

PikaPlanner v4

대회 당일 스킬 적용 · 실험 · 제출 실전 가이드

읽고 위에서부터 따라가면 1 시간 안에 안전한 제출본을 만드는 것을 목표로 한다.

이 문서가 약속하는 것

어려운 말은 먼저 뜻을 설명한다. 당일에는 공개 소스와 제공 봇을 먼저 읽고, v4 의 안정적인 기본 플레이를 보존한 채 필요한 스킬 부분만 붙인다. 한 번에 하나씩 바꾸고, 같은 조건으로 비교하고, 마지막에는 반드시 되돌릴 수 있는 기준본을 남긴다.

현재 v4 는 네트 앞 급강하 방어, 긴 랠리 5 터치 안정화, 입력 지연/코트 경계 수정, 장기 반복 랠리 탈출, 라운드 전환 정지 상태 처리까지 포함한 최신 기준본이다.

0. 가장 먼저 읽는 2 페이지

0.1 당일 1 시간을 한눈에 보기

00:00 v4 BASE 복사 · 새 repo에서 정상 실행 확인
00:05 새 snapshot / gauge / activation / 제공 skill bot 읽기
00:15 스킬을 딱 한 번 쓰는 초소형 테스트
00:25 상대 스킬을 먼저 이해시키기: reach / threat / ball physics
00:35 우리 스킬 후보 추가
00:45 A/B 테스트: 제공 skill bot + 기존 회귀 상대
00:53 한두 값만 튜닝
00:57 코드 동결 · 검증 · 제출

가장 중요한 원칙

당일의 목표는 “스킬을 많이 쓰는 봇”이 아니다. “기존 v4의 실수를 거의 하지 않는 플레이를 유지하면서, 가치 있는 순간에만 스킬을 쓰고 상대의 스킬도 늦지 않게 막는 봇”이다.

0.2 파일을 이렇게 쓴다

파일	용도
PikaPlanner_v4_BASE.js	절대 수정하지 않는 보험. 스킬 패치가 망하면 이쪽으로 돌아간다.
PikaPlanner_v4_SKILL_WORKING_COPY.js	당일 실제로 수정하는 작업본.
SKILL_FACTS_WORKSHEET.md	코딩 전에 공개 스킬의 사실을 적는 종이.
MATCHDAY_60MIN_CHECKLIST.md	시간이 부족할 때 이 파일만 보고 진행.
TEST_LOG_TEMPLATE.md	후보 A/B 결과와 실패 유형을 기록.

0.3 코드에서 당일 주로 볼 곳

```
const SKILL = {  
  read(...) // 새 snapshot 필드 읽기  
  ballFrameHook(...) // 스킬이 공 물리를 바꾸면  
  reachOverride(...) // 대시/텔레포트/슈퍼점프/히트박스 변화  
  extraOpponentThreats(...) // 상대 스킬 공격을 미리 수비기에 알려주기  
  extraActions(...) // 우리가 쓸 스킬 후보 추가  
  evaluate(...) // 게이지를 쓸 가치 조정  
  emergencyOverride(...) // 아주 좁은 구조용 예외  
  decorateAction(...) // API가 실제로 새 반환 필드를 요구할 때만  
};
```

당일 공개 소스가 이 중 일부만 필요하다고 말하면 그 일부만 채운다. 나머지는 그대로 빈 함수로 둔다.

1. 대회 규칙이 코드 선택에 주는 영향

1.1 10 점 선착제 + 4 분 제한

정상적으로는 10 점을 먼저 얻으면 세트가 끝난다. 하지만 4 분이 되면 그때 앞선 팀이 바로 이긴다. 4 분 동점이면 5 분까지 다음 한 점을 먼저 얻는 골든볼이고, 5 분에도 동점이면 무승부다.

왜 v4 의 장기 랠리 탈출 장치를 유지하는가

900 rally frame 은 약 36 초다. 한 랠리가 36 초 동안 같은 궤도를 반복하면 4 분 세트의 약 15%를 한 점도 없이 소모한다. v4 는 아주 긴 랠리에서, 상대 접촉이 임박하지 않았을 때만 수비 대기 위치를 약 6px 바뀔 반복을 깨도록 해 두었다.

반대로 “지금 앞서 있으니 시간을 일부러 끌자” 같은 별도 시간 공략은 v4 에 넣지 않았다. 현재 snapshot 에 공식 남은 시간이 명시되어 있다는 보장이 없기 때문이다. 당일 새 repo 가 정확한 타이머 필드를 제공하지 않는다면 이 부분은 추측으로 건드리지 않는 편이 안전하다.

1.2 예선은 한 경기 한 세트, 팀당 6 경기

한 세트만 치르므로 한 번의 자살, 5 터치, 좌우 한쪽 버그가 곧 경기 패배로 이어질 수 있다. 그래서 평균 공격력 조금 올리는 것보다 “말도 안 되는 한 점”을 줄이는 게 중요하다.

조 3 위 비교에는 승점과 득실차가 사용된다. 승리가 가장 중요하지만, 같은 승수라면 10-2 처럼 큰 점수차가 10-9 보다 유리할 수 있다. 즉 안정적으로 이기면서 공짜 실점을 줄이는 방향이 맞다.

1.3 본선은 3 세트, 2 세트 선취

본선에서는 한 세트의 우연이 조금 희석된다. 그래도 당일 한 시간 안에 만든 미검증 스킬 트릭보다, v4 의 기본기를 유지한 단순한 스킬 사용이 더 좋은 기본 선택이다.

1.4 첫 서브는 랜덤

서브가 누가 먼저냐에 따라 랠리의 첫 패턴이 달라질 수 있다. 따라서 한 seed 에서 한쪽만 이겼다고 바로 결론내리면 안 된다. 최소한 LEFT/RIGHT 와 두 개 이상의 seed 를 본다.

중요한 시뮬레이터 수정

이전 로컬 harness 는 포인트마다 서브를 다시 랜덤으로 뽑는 부분이 있었다. 대회 규칙을 검토하면서 이를 수정했다. 이제 main simulator 는 첫 서브만 랜덤으로 정하고, 이후는 원작 흐름을 따른다. 예전 scoreline 은 “버그 재현/회귀 신호”로는 유효하지만 실제 대회 점수 예측으로 다시 사용하지 않는다.

1.5 5 번째 같은 진영 접촉은 즉시 실점

스킬이 이동만 바뀌도 공을 두 번 스치거나, 자기 코트에 공을 다시 남기는 접촉이 늘 수 있다. 따라서 스킬 테스트에서 “몇 번 썼는가”뿐 아니라 “한 점에서 몇 번 우리 쪽 접촉이 생겼는가”도 본다.

2. 용어 사전 - 이 정도만 알면 된다

용어	이 문서에서 뜻하는 것
snapshot / 스냅샷	decide()가 받을 현재 상태 묶음. 공, 두 플레이어, 점수, 추가 스킬 필드 등이 들어간다.
frame / 프레임	게임 물리가 한 번 진행되는 가장 작은 시간 단위. 25 FPS 라서 한 프레임은 40ms.
decision / 결정	봇이 snapshot 을 보고 `{x,y,hit}`를 한 번 계산하는 것. 현재 3 프레임 마다 새 결정을 요청한다.
action latency / 입력 지연	계산한 행동이 실제 물리에 적용되기까지의 지연. v4 기본 모델은 빠른 JS 환경의 1 프레임을 기준으로 한다.
trajectory / 궤적	공이 앞으로 거칠 위치를 프레임 순서대로 계산한 목록.
reach / 도달 가능	몇 프레임 뒤 공 위치에 캐릭터의 몸이 실제로 닿을 수 있는가.
hitbox / 히트박스	공과 캐릭터가 충돌했다고 판정되는 범위. 기본 플레이어는 중심에서 상하좌우 32px.
candidate / 후보	봇이 비교해 보는 행동 선택지. 일반 공격, 안전 리시브, 스킬 공격 등이 후보가 될 수 있다.
score / 내부 평가점수	후보 중 무엇이 나은지 비교하기 위한 코드 내부 숫자. 경기 스코어가 아니다.
baseline / 기준본	수정 전 안정적인 비교 대상. 여기서는 v4 BASE.
A/B test	A 는 기존 후보, B 는 한 가지를 바꾼 후보로 두고 같은 조건에서 비교하는 실험.
regression / 회귀 테스트	새 수정이 과거에 고쳤던 문제를 다시 만들지 않는지 확인하는 테스트.
seed / 시드	랜덤 시작 상태를 재현하기 위한 숫자. 같은 seed 는 같은 초기 난수 흐름을 만들기 쉽다.
mirror / 미러	LEFT 상황을 RIGHT 로 정확히 뒤집었을 때 판단도 반대로 뒤집히는지 확인하는 것.
frozen round / 라운드 정지 구간	득점 후 다음 랠리 준비 동안 decide 는 계속 호출되지만 실제 물리는 잠시 멈춘 상태.
no-op	함수 자리는 있지만 지금은 아무 효과도 내지 않는 상태.

모르는 단어가 나오면

영어 단어를 외우는 게 목적이 아니다. “이 값이 공을 바꾸는가?”, “플레이어가 더 빨리 달게 하는가?”, “스킬을 언제 쓸 수 있는가?” 이 세 질문으로 다시 풀어 생각하면 된다.

3. v4 는 평소에 어떻게 플레이하는가

3.1 머릿속 흐름

[현재 상태 읽기]
↓
[공이 앞으로 어디로 갈지 계산]
↓
[상대가 먼저 공을 칠 수 있으면 그 공격까지 미리 계산]
↓
[내가 받을 위치 / 점프 / 다이빙 후보 계산]
↓
[여러 공격을 실제로 날려보고 상대가 받을 수 있는지 비교]
↓
[가장 좋은 행동을 안전 실행기로 통과시켜 반환]

핵심은 “공을 따라간다”가 아니라 “몇 프레임 뒤 누가 어디에서 공에 닿을지 계산한다”는 점이다. 그래서 새 스킬이 접촉 시간을 바꾸는 순간 v4 의 수비 모델도 같이 고쳐야 한다.

3.2 지금까지 발견한 실제 실수와 v4 의 해결

과거 실패	v4 의 처리
네트 앞에서 공이 막 넘어오자마자 수직 급강하를 맞음	공이 아직 우리 쪽이어도 상대가 곧 네트 너머에서 칠 수 있으면 위협 탐색을 계속한다.
수비 후보가 모두 “못 막음”이라 현재 위치에 멈춤	몇 px 모자라는지까지 점수에 반영해 가장 덜 부족한 위치를 고른다.
바닥 판정 프레임의 공을 막을 수 있다고 착각	엔진은 world/ground 판정을 먼저 하므로 ground frame 은 이미 늦은 것으로 본다.
천천히 눈앞에 떨어지는 공을 놓침	현재 위치에서 내린 결정을 미래 위치에서 다시 내린 것처럼 계산하던 sample-and-hold 오차를 수정했다.
네트 경계 밖의 공을 닿을 수 있다고 판단해 헛점프	실제 플레이어 x 경계로 이동 가능 범위를 잘라낸다.
긴 랠리에서 ordinary touch 가 누적되어 5 터치	world-only 공 예측과 실제 공을 비교해 접촉을 추정하고, 위험한 로컬 터치를 강하게 피한다.
득점 후 정지 snapshot 이 반복되며 touch 추정이 증가하거나 stale jump 가 남음	정확한 round-reset geometry 를 frozen state 로 감지해 touch 추정을 멈추고 jump/hit 을 억제한다.

3.3 무엇을 함부로 바꾸지 말아야 하나

W=432, NET_X=216, PLAYER_HALF=32, WALK=6, JUMP_VY=-16, GRAVITY=1 같은 값은 공개 엔진 규칙이다. 당일 스킬 소스가 실제로 이 값을 바꾼다는 근거가 없으면 수정하지 않는다.

반면 게이지 최대치, 한 번 사용 비용, 대시 거리, 스킬 지속 프레임, 특수 공 속도는 당일 새로 공개되는 값이므로 별도의 SKILL 설정으로 넣는 것이 좋다.

4. 스킬 공개 직후: 코딩보다 먼저 “사실”을 모은다

4.1 운영진 Q&A 가 이미 알려준 것

- 기존 snapshot 필드는 유지되고 새 필드만 추가된다.
- 히든 룰은 없고 스킬만 추가된다.
- 첫 경기 전 1시간이 주어진다.
- 스킬을 포함한 새 repository 를 localhost 에서 실행할 수 있다.
- 기본 AI 는 스킬을 못 쓰지만, 일정 수준의 스킬 사용 봇 코드가 따로 제공된다.
- 새 스킬은 이론적으로 충분히 방어 가능하다.
- 같은 조건에서 반복되는 이상 충돌은 “게임의 일부”로 처리된다.
- LLM 질문은 팀당 3 회, 회당 300 자 제한이다.

이 정보가 주는 결론

우리의 게이지+대시/텔레포트 예상은 준비용 가설일 뿐이다. 당일에는 새 repo 의 실제 코드와 제공 skill bot 이 정답이다. 소스를 10 분 읽고 나서 구현하는 것이 추측으로 30 분 코딩하는 것보다 빠르다.

4.2 10 분 동안 검색할 키워드

skill gauge charge cooldown ready ability
dash teleport speed jump state hitbox
snapshot decide action input collision physics

4.3 제공 skill bot 에서 반드시 뽑을 것

1. 어떤 snapshot 필드를 읽는가.
2. 어떤 조건에서 스킬을 “사용 가능”이라고 보는가.
3. decide 반환값에 기존 x/y/hit 외에 새 필드가 있는가.
4. 지상/공중/파워 준비 상태 중 어디서 쓰는가.
5. 스킬 사용 전 어떤 위치로 이동하는가.
6. 사용 후 공이나 플레이어 상태가 어떻게 변하는가.
7. 상대 스킬을 막는 코드가 있다면 어떤 조건을 보는가.

4.4 3 번뿐인 LLM 질문은 어디에 쓰나

코드에서 직접 확인되는 내용을 묻지 않는다. 3 번은 “소스를 봤는데도 애매한 것”을 정리하는 데 쓴다. 예를 들어 발동 요청 시점과 게이지 차감 시점이 다른지, 이동 중 collision 을 검사하는지, 제공 봇의 특정 조건문이 무엇을 방어하는지 같은 질문이다.

5. 60 분 작업 순서 - 그대로 따라 하기

5.1 00:00-00:05 - 기준본을 잠근다

8. 'PikaPlanner_v4.js'를 'PikaPlanner_v4_BASE.js'로 복사한다.
9. 새 repository 에서 BASE 를 한 번 돌린다.
10. 기존 '{x,y,hit}'만 반환해도 정상 경기가 되는지 확인한다.
11. BASE 는 이후 절대 수정하지 않는다.

이 단계의 목적은 “스킬 패치가 망해도 출전 가능한 파일”을 확보하는 것이다.

5.2 00:05-00:15 - 코딩하지 말고 worksheet 를 채운다

'SKILL_FACTS_WORKSHEET.md'를 열고 snapshot, activation, frame order, gauge, mobility 항목을 실제 소스에서 채운다. 모르는 칸은 빈 칸으로 두고 추측하지 않는다.

다음 단계로 넘어가는 최소 조건

스킬을 사용할 수 있는 조건, 발동 방법, 게이지 비용, 플레이어/공 중 무엇이 바뀌는지, 상대가 쏠 때 우리가 관측할 수 있는 정보가 무엇인지 설명할 수 있어야 한다.

5.3 00:15-00:25 - 스킬을 한 번만 쓰는 미니 봇

강한 전략을 만들기 전에 “스킬이 실제로 어떤 프레임에서 무엇을 바꾸는지”를 측정한다.

```
let used = false;

function decide(s) {
  if (!used && <스킬 사용 가능>) {
    used = true;
    return <공개 API 의 정확한 스킬 발동 action>;
  }
  return {x:0, y:0, hit:0};
}
```

대시라면 매 프레임 player.x 를 기록한다. 텔레포트라면 시작점-끝점 사이에서도 collision 을 검사하는지 본다. 공을 바꾸는 기술이면 skill 처리 전후의 vx/vy 와 world/net/ground 처리 순서를 적는다.

5.4 00:25-00:45 - 방어를 먼저, 공격을 다음에

```
[새 필드 읽기]
↓
[스킬 발동 contract 확인]
↓
[공 물리 / 상대 도달거리 수정]
↓
[상대가 만들 새 위협 추가]
↓
[우리 스킬 후보 추가]
↓
[게이지를 언제 쏠지 튜닝]
```

이 순서를 지키는 이유는 간단하다. 우리 공격만 먼저 붙이면, 상대가 스킬로 예상보다 5 프레임 빨리 공을 쳐도 v4 는 여전히 옛 접촉 시간을 믿는다. 그러면 네트 앞 급강하 때와 같은 “수비가 한 박자 늦는” 문제가 다시 생긴다.

5.5 00:45-01:00 - A/B, 한두 값만 튜닝, 동결

후보 A 를 남기고 후보 B 에서 한 가지만 바꾼다. 같은 상대/side/seed 에서 비교한 뒤 좋아진 이유가 설명되면 seed 와 side 를 넓힌다. 57 분이 되면 구조 변경을 중지한다.

6. 스킬을 2 분 안에 분류하는 방법

아래 질문을 위에서부터 답하면 어느 hook 을 고칠지 정할 수 있다.

질문	YES 일 때 먼저 볼 곳
snapshot 에 gauge / ready / cooldown 같은 새 정보가 있는가?	SKILL.read
스킬 발동 때문에 decide 의 반환 형식이 실제로 바뀌는가?	SKILL.decorateAction - 단, 업데이트된 API 가 명시한 경우만
스킬이 공의 속도, 중력, 방향, 반사를 바꾸는가?	SKILL.ballFrameHook
스킬이 플레이어를 더 빨리/멀리 이동시키거나 점프/히트박스를 바꾸는가?	SKILL.reachOverride
상대가 스킬로 기존에 없던 공격 궤적을 만들 수 있는가?	SKILL.extraOpponentThreats
우리가 선택할 새 행동이 생기는가?	SKILL.extraActions
스킬을 아껴야 하는가 / 게이지 사용 가치가 상황마다 다른가?	SKILL.evaluate
지금 안 쓰면 실점이고 쓰면 확실히 사는 좁은 구조 상황이 있는가?	SKILL.emergencyOverride

6.1 가장 많이 나올 세 가지 조합

스킬 유형	필수 연결
대시 / 텔레포트	read → reachOverride → extraOpponentThreats → extraActions → evaluate
특수 스매시 / 공 가속	read → ballFrameHook 또는 특수 contact 모델 → extraOpponentThreats → extraActions
슈퍼점프 / 공중 이동	read → reachOverride → extraOpponentThreats → extraActions

6.2 “뭔지 모르겠다”면 다섯 문장으로 다시 본다

- 언제 사용할 수 있는가?
- 사용하면 플레이어 위치/상태가 어떻게 변하는가?
- 사용하면 공 위치/속도가 어떻게 변하는가?
- 상대도 같은 기술을 쓸 때 우리가 언제 처음 맞을 수 있는가?
- 한 번 쓰면 얼마만큼의 자원(게이지/쿨다운)을 잃는가?

7. v4 에 스킬을 붙이는 실제 코드 순서

Step 1. SKILL.read - 새 필드를 “한 번만” 번역한다

실제 snapshot 필드 이름을 planner 전체에 흩뿌리지 않는다. read 에서 우리가 쓰기 쉬운 이름으로 바꾼다.

```
read(snapshot) {
  return {
    enabled: true,
    selfGauge: snapshot.<실제 필드>,
    oppGauge: snapshot.<실제 필드>,
    ready: snapshot.<실제 필드>,
    cooldown: snapshot.<실제 필드>,
  };
}
```

이렇게 하면 이후 코드는 'c.skill.selfGauge'만 보면 된다. 필드 이름이 바뀌거나 구조가 중첩되어도 read 만 고치면 된다.

Step 2. decorateAction - 정말 새 반환 필드가 있을 때만

현재 공개 규칙은 '{x,y,hit}'를 요구한다. 당일 API 가 “skill: 1” 같은 새 반환 필드를 명시할 때만 이 함수를 쓴다.

```
decorateAction(ctx, planned, base) {
  if (planned.useSkill) {
    return {...base, <실제 새 필드>: <실제 값>};
  }
  return base;
}
```

금지

“아마 skill:true 겠지”라고 추측해서 반환 필드를 추가하지 않는다. 제공 skill bot 의 실제 return 과 업데이트된 API 를 그대로 복사한다.

Step 3. ballFrameHook - 공의 물리가 바뀌면 한 곳에 정확히

공의 vx/vy, 중력, 네트/벽 반사, curve 가 바뀌는 기술이라면 처리 순서를 released source 그대로 넣는다.

```
ballFrameHook(ball, skillState, events) {
  // 새 repo 의 실제 프레임 순서대로만 작성
  // 예: ball.vx = ...; ball.vy += ...;
}
```

공 계산을 공격/수비마다 따로 복사하면 한쪽만 고쳐지고 다른 쪽이 틀릴 가능성이 높다. v4 는 이 hook 을 공 시뮬레이터 한 곳에서 호출하도록 해 두었다.

Step 4. reachOverride - “언제 닿을 수 있나”가 바뀌면 가장 중요

대시, 텔레포트, 슈퍼점프, 이동속도 증가, 히트박스 확대가 여기에 해당한다. 먼저 상대 쪽 도달 가능성을 고친다.

```
reachOverride(ctx, who, query, baseResult) {  
  if (who !== "opp") return null;  
  if (!<상대 스킬 사용 가능>) return null;  
  
  const skillCanReach = <정확한 대시/점프/텔레포트 계산>;  
  if (!skillCanReach) return null;  
  return {can:true, options:baseResult.options + 1};  
}
```

왜 상대부터인가

상대가 원래 8 프레임 뒤에만 닿던 공을 스킬로 3 프레임 뒤에 칠 수 있다면, 우리 수비는 5 프레임 일찍 준비해야 한다. 이 부분을 안 고치고 우리 스킬 공격만 넣으면 공격은 강해져도 수비가 다시 늦어진다.

Step 5. extraOpponentThreats - 새 공격 궤적을 수비기에 보여준다

```
extraOpponentThreats(ctx, oppVy) {  
  if (!<상대가 스킬을 쓸 수 있음>) return [];  
  const threats = [];  
  // 1) skill contact 가능한 시점 계산  
  // 2) contact 후 ball 상태 생성  
  // 3) simulateTrajectory로 끝까지 진행  
  // 4) {type, contactFrame, tr, landing} 추가  
  return threats;  
}
```

처음에는 상대에게 약간 유리한 쪽으로 잡아도 된다. “쓸 수도 있는데 실제로 안 쓰는 위협”이 조금 포함되는 것보다 “실제로 가능한 급강하를 목록에서 빼먹는 것”이 더 위험하다.

Step 6. extraActions - 이제 우리 스킬을 후보로 만든다

```
extraActions(ctx, helpers) {  
  if (!<우리 스킬 사용 가능>) return [];  
  const out = [];  
  // 스킬을 썼을 때 contact와 ball 상태를 정확히 계산  
  // helpers.simulateTrajectory / evaluateOutgoingTrajectory 재사용  
  out.push({  
    type: "SKILL_ATTACK",  
    x: ..., y: ..., hit: ...,  
    useSkill: true,  
    score: <기존 공격과 비교 가능한 값>  
  });  
  return out;  
}
```

스킬 후보는 기존 POWER/CLEAR/DEFEND와 경쟁한다. 단순히 “스킬 score가 1 이상”이라고 무조건 스킬을 쓰면 안 된다.

Step 7. evaluate - 게이지를 아끼는 규칙은 마지막에

상황	처음 권장
스킬 없으면 실점, 쓰면 확실히 살림	거의 항상 사용
스킬 공격이 일반 공격보다 명확히 강함	사용
일반 공격과 비슷함	아낌
게이지가 최대치 근처라 추가 충전이 낭비됨	사용 가치 조금 증가
스킬 후 위험한 위치/긴 회복 상태	강하게 감점

8. 예측이 맞았을 때: 게이지 + 대시/텔레포트

사전에 가장 가능성을 높게 본 경로다. 정확히 맞았다면 이 장을 순서대로 따른다. 맞지 않아도 “이동 능력이 바뀌는 스킬”에는 거의 그대로 쓸 수 있다.

8.1 가장 먼저 적을 숫자

번호	확인할 것
1	게이지 최대치
2	한 번 사용 비용
3	게이지가 차는 조건
4	포인트가 끝나면 유지/리셋 여부
5	대시 거리 또는 속도
6	대시 지속 프레임
7	지상/공중 사용 가능 여부
8	사용 후 recovery/lockout
9	이동 중 공과 충돌하는지, 끝점에서만 충돌하는지
10	대시 중 hit/power 상태를 같이 유지할 수 있는지

8.2 대시와 텔레포트는 같은 식으로 계산하지 않는다

대시	텔레포트
여러 프레임 동안 중간 위치를 지나갈 수 있다.	중간 위치가 존재하지 않을 수 있다.
이동 도중 ball collision 이 생길 수 있다.	끝점에서만 collision 인지 별도 확인해야 한다.
방향/속도/지속시간이 핵심이다.	목적지 결정과 clamp 규칙이 핵심이다.
매 프레임 reach 를 확장하는 모델이 자연스럽다.	특정 프레임에 새 위치 집합을 추가하는 모델이 자연스럽다.

8.3 구현 순서

1. read: gauge / ready / cooldown 읽기
2. 스킬을 한 번만 발동하는 micro-test
3. 프레임별 실제 이동 측정
4. reachOverride: 상대 접촉 시간이 얼마나 빨라지는지 반영
5. extraOpponentThreats: 대시 후 급강하/특수 contact 추가
6. extraActions: 우리 대시 공격 또는 rescue 후보 추가
7. evaluate: 게이지 낭비 줄이기

8.4 첫 버전은 두 경우만 써도 된다

- 일반 이동으로는 못 닿는 공인데, 스킬을 쓰면 확실히 닿고 네트를 넘길 수 있다.
- 스킬 후 공격의 평가가 일반 공격보다 “확실히” 좋다.

이 정도만 구현해도 상대 팀들이 새 테마에 익숙하지 않다면 충분히 실전적인 이득을 만들 가능성이 있다. 처음부터 복잡한 게이 지 경제나 3수 앞 콤보를 만들 필요가 없다.

9. 예측과 다른 스킬이 나왔을 때 빠른 경로

9.1 공이 갑자기 빨라지거나 꺾인다

가장 먼저 `ballFrameHook`을 정확히 만든다. 공 시뮬레이터가 맞아야 공격도 수비도 맞는다. 그 다음 상대 skill threat 와 우리 skill candidate 를 같은 시뮬레이터로 만든다.

9.2 강화 스매시 / 특수 타격

접촉 가능한 시점은 그대로인데 접촉 후 공 속도만 달라진다면 `reachOverride`보다 “새 contact 후 ball 상태”가 먼저다. 일반 POWER 후보 옆에 특수 타격 후보를 만들고 기존 trajectory 평가기로 비교한다.

9.3 슈퍼점프 / 2 단 점프

핵심은 상대가 더 높은 공을 더 일찍 칠 수 있다는 것이다. `reachOverride`로 상대 접촉 시간을 먼저 넓힌 뒤, 우리 사용은 `extraActions`에서 만든다.

9.4 히트박스 확대

충돌 반경이 32 에서 다른 값으로 바뀌는지, 효과가 몇 프레임 지속되는지 확인한다. 상대가 더 멀리 있는 공에 닿을 수 있으므로 reach 쪽이 우선이다.

9.5 실드 / 방어 스킬

정말 방어 전용이라면 `emergencyOverride`가 편할 수 있다. 하지만 스킬이 공을 튕기거나 속도를 바꾸면 그 후의 공 물리도 반드시 `ballFrameHook` 또는 별도 skill threat/contact 모델에 반영한다.

9.6 완전히 다른 스킬

새 snapshot 정보?	→ read
새 decide 반환 필드?	→ decorateAction
공 물리 변화?	→ ballFrameHook
누가 언제 공에 닿는지 변화?	→ reachOverride
상대의 새 공격 궤적?	→ extraOpponentThreats
우리의 새 행동?	→ extraActions
언제 자원을 쓸지?	→ evaluate

10. 테스트 - “이겼다”가 아니라 “왜 이겼고 왜 졌는지”

10.1 테스트 사다리

12. 문법/반환값: decide 가 항상 유효한 action 을 내는가.
13. 스킬 단위 실험: 발동/게이지/물리가 새 repo 와 정확히 같은가.
14. 제공 skill bot: 상대 스킬을 너무 늦게 인식하지 않는가.
15. 제공 skillless AI: 스킬이 없을 때 기존 기본기가 망가지지 않았는가.
16. v05/v07: 네트 앞 빠른 공격 방어가 유지되는가.
17. v2/v2.1/tuned v1: 긴 랠리에서 느린 공 미스, 5 터치, 헛점프가 다시 안 나오는가.
18. LEFT/RIGHT: 한쪽에서만 망가지는가.
19. 여러 seed: 한 전개에만 우연히 맞는가.
20. 10 점/4 분 tournament mode: 시간 제한에서 이상한 무승부/장기 랠리가 생기는가.

10.2 꼭 따로 세는 실패 유형

실패	보통 먼저 의심할 곳
자기 코트에 파워샷을 꽂음	contact 후 ball physics / shot scoring / skill ball model
5 터치	touch inference / local body return / skill multi-contact
천천히 눈앞에 떨어지는 공을 놓침	sample-and-hold timing / first contact / court boundary
네트 앞 급강하를 늦게 봄	opponent reach / extraOpponentThreats / reaction timing
불가능한 공에 점프/다이빙	reach 계산 / hitbox / skill state restriction
스킬 버튼이 가끔 안 먹힘	activation state / gauge consumption / action application timing
상대 스킬을 맞고서야 반응	reachOverride 또는 skill threat 가 planner 에 연결되지 않음
득점 직후 이상한 jump/dive	frozen-round guard / held input
매우 긴 랠리 0-0 반복	long-rally loop / deterministic orbit

10.3 seed 와 side 결과를 과해석하지 않는다

같은 seed 에서 봇의 좌우만 바꾸면 P1/P2 와 첫 서브 주체가 달라질 수 있다. 따라서 “LEFT 10-0, RIGHT 4-10”만 보고 코드 비대칭이라고 단정하지 않는다. 코드 자체의 대칭은 `mirror\_sanity.mjs`로 확인하고, 실제 성적 차이는 별도로 replay 를 본다.

10.4 현재 simulator 의 위치

패키지 simulator 는 버그 재현과 후보 비교용이다. 이번 규칙 검토에서 “매 포인트 서브 랜덤” 오류를 고쳤고, 10 점/4 분/골든 볼 및 round freeze 를 더 가깝게 반영했다. 그래도 당일 새 repository 와 실제 브라우저 worker timing 이 최종 정답이다.

11. 튜닝 - 한 번에 하나만 바꾸기

11.1 가장 안전한 실험 방법

후보 A = 지금까지 가장 안정적인 파일
↓
딱 한 가지 값/조건만 바꾼 후보 B 생성
↓
같은 상대 + 같은 side + 같은 seed 로 비교
↓
승패 + 점수차 + 실패 유형 기록
↓
좋아진 이유가 설명되면 반대 side / 다른 seed 추가
↓
유지 또는 즉시 revert

11.2 당일 튜닝 우선순위

우선순위	튜닝 대상
1	스킬이 실제로 발동하는 correctness
2	상대 스킬 접촉 시간을 놓치지 않는 defense
3	자살/5 터치/헛점프 같은 치명적 실패 제거
4	게이지를 너무 일찍/너무 늦게 쓰는 임계값
5	스킬 공격 후보 수 / contact timing 세부 최적화
6	기존 v4 의 수십 개 일반 가중치 재튜닝 - 당일에는 웬만하면 하지 않음

11.3 상대 수준이 아주 높지 않을 가능성을 어떻게 반영하나

v05/v07/v2 는 우리의 stress test 다. 실제 참가자들이 전부 이런 수준이라고 가정하지 않는다. 따라서 “이 강한 봇 하나만 잡는 특수 대응”보다 다음을 우선한다.

- 평범한 공을 거의 놓치지 않기.
- 자기 코트 자살과 5 터치를 피하기.
- 제공 skill bot 의 대표 공격을 방어하기.
- 스킬이 확실히 좋은 순간에만 간단하게 사용하기.
- LEFT/RIGHT 둘 다 정상 플레이하기.

이 다섯 가지가 되면 복잡한 3 수 앞 스킬 콤보보다 실전 기대값이 높을 수 있다.

11.4 5-0 한 번을 과신하지 않는다

한 seed 에서는 수정 하나가 굉장히 좋아 보일 수 있다. 반대 side 와 다른 seed 를 추가한다. 상대가 `Math.random()` 을 내부에서 쓰면 같은 game seed 라도 세부 수비 움직임이 달라질 수 있다.

12. 문제가 생겼을 때 따라가는 디버깅 순서

12.1 “스킬을 쓰고 졌다”면

21. 스킬이 실제로 요청된 프레임을 찾는다.
22. 그 프레임의 gauge/ready/state 가 조건과 맞았는지 본다.
23. 다음 1~10 프레임의 player.x/y/state 와 ball.x/y/vx/vy 를 본다.
24. 우리 내부 예측과 실제 엔진 상태가 처음 달라지는 프레임을 찾는다.
25. 그 차이가 reach 인지 ball physics 인지 action timing 인지 분류한다.
26. 분류한 함수 하나만 고친다.

12.2 “스킬을 안 썼는데 기존 플레이가 망가졌다”면

가장 먼저 BASE 와 diff 를 본다. `read()`만 추가했는데 행동이 바뀌었다면 skill state 의 기본값이 planner 에 잘못 들어갔을 가능성이 있다. `skill unavailable` 상태에서는 새 hook 들이 모두 no-op 이 되게 만든다.

12.3 “상대 스킬을 항상 한 박자 늦게 막는다”면

우리 skill attack score 를 만지지 말고 먼저 상대 접촉 시점을 본다. 일반 reach 로 8 프레임 뒤라고 계산했는데 실제 skill 은 3 프레임 뒤 접촉하는지 확인한다. 맞다면 `reachOverride` 또는 `extraOpponentThreats`가 부족한 것이다.

12.4 “오래 할수록 이상해진다”면

전역 memory 가 실제 사건보다 한 번씩 더 증가하는지 본다. 특히 score reset, net crossing, frozen round, skill contact, multi-contact 를 확인한다. 긴 랠리 버그는 평균 승률보다 replay 한 장면이 더 많은 정보를 준다.

12.5 “한쪽 코트에서만 이상하다”면

첫째 mirror sanity 를 돌린다. 통과하면 내부 좌우 변환은 대체로 정상이다. 둘째 첫 서브/서버 identity/특정 rally route 가 다른지 replay 를 비교한다. 코트별 점수 차이가 곧 코드 비대칭은 아니다.

13. LLM 질문 3 회를 아끼는 방법

팀당 3 회, 300 자 제한이므로 “코드에서 바로 보이는 정보”를 묻지 않는다. 새 repo 를 먼저 읽고, 애매한 프레임 순서나 특정 bug path 를 압축해서 묻는다.

질문 후보 1 - API 계약

“새 스킬 관련 소스 기준으로 추가 snapshot 필드, 게이지 증가/소비/리셋, 발동 action 형식, 사용 불가 상태를 파일/함수명과 함께 요약해줘. 추측 말고 실행되는 코드만.”

질문 후보 2 - 정확한 프레임 순서

“스킬 발동 프레임의 처리 순서를 코드 기준으로 알려줘: ball world physics, ordinary move, skill move, player-ball collision, ground/score, skill state update. dash/teleport 중간 경로 collision 여부도 포함.”

질문 후보 3 - 실제 실패 장면

“이 replay 에서 우리 예측과 실제 상태가 처음 달라지는 프레임을 찾고 원인을 reach/ball physics/action timing/state 중 하나로 분류해줘. 수정해야 할 최소 함수와 이유만 답해줘.”

질문 작성 원칙
이미 아는 사실은 질문에서 뺀다. 파일명/함수명/문제 프레임을 넣는다. “최강 전략 짜줘”보다 “이 코드 순서가 정확히 무엇인가”가 1 시간 대회에서 훨씬 가치 있다.

14. 제출 직전 3분 체크리스트

27. 제출 파일이 BASE 가 아니라 최종 후보인가.
28. 파일 하나의 '.js'이고 import/require 가 없는가.
29. 4 MB 보다 충분히 작은가.
30. 'decide(snapshot)'이 모든 상황에서 유효한 action 을 반환하는가.
31. 새 API 가 action 확장을 명시하지 않았다면 반환은 '{x,y,hit}' 그대로인가.
32. 게이지 0 / skill unavailable 에서도 정상 경기되는가.
33. 게이지가 충분할 때 스킬이 실제로 최소 한 번 발동하는가.
34. LEFT 와 RIGHT 를 각각 확인했는가.
35. 제공 skill bot 의 대표 공격을 한 번 이상 막아봤는가.
36. 자기 코트 자살 / 5 터치 / 느린 공 미스 / round-reset stale jump 가 없는가.
37. 최근 변경이 하나라면 바로 전 후보와 diff 를 눈으로 봤는가.
38. 평균/최대 decide 시간이 120ms 목표에 넉넉한가.
39. 57 분 이후에 큰 구조 변경을 하지 않았는가.

최종 revert 규칙

50 분이 넘었는데 skill patch 가 여전히 불안정하면 깨끗한 v4 BASE 를 제출하는 쪽이 낫다. 운영진은 기존 snapshot 필드가 유지되고 스킬을 무시한 봇도 정상 진행된다고 명시했다. “스킬을 못 쓰는 안정적 봇”은 “스킬 때문에 자살하는 봇”보다 낫다.

최종 파일명

sha ims 제출 페이지에서는 자동 규약 저장이 되지만, 로컬 'src/code-here/' 테스트에서는 '〈팀명〉_v〈버전〉.js' 형식을 맞춰야 드롭다운에 뜬다. 최종본을 복사해 팀명과 버전을 넣는다.

부록 A. v4 함수 지도

함수/영역	무엇을 하는 곳
canonicalize	LEFT/RIGHT 를 내부 LEFT 기준으로 통일한다.
isRoundFrozenState	득점 후 physics 가 멈춘 reset snapshot 을 구분한다.
updateMemory	점수/네트 crossing/접촉 추정/self-set 상태를 기억한다.
stepBallWorld	벽/천장/네트/바닥 + skill ball hook 을 한 프레임 계산한다.
simulateTrajectory	공을 여러 프레임 앞으로 굴러 궤적을 만든다.
canPlayerReachBallAt	특정 시간/위치의 공에 플레이어가 닿을 수 있는지 계산한다. skill reach hook 이 연결되어 있다.
evaluateOutgoingTrajectory	우리 공격이 상대에게 얼마나 받기 어려운지 점수화한다.
choosePowerAttack	공중 파워히트 후보를 비교한다.
findBestGroundReceive	지상에서 실제 첫 접촉을 기준으로 리시브를 고른다.
findJumpTakeoff	새 점프를 시작할지 계산한다.
findAirborneIntercept	이미 공중일 때 접촉할 공을 찾는다.
predictOpponentThreats	상대가 언제 먼저 칠지 보고 가능한 공격 궤적을 만든다.
chooseDefensiveStandby	여러 상대 공격을 최대한 함께 커버할 수비 위치를 고른다.
planCore	POWER → 방어 예측 → 리시브/점프/다이빙 등 큰 판단 순서를 정한다.
executeSafely	stale hit/jump 가 landing 후 dive/rejump 가 되지 않도록 마지막 안전검사를 한다.
SKILL.*	당일 스킬을 좁게 연결하는 전용 영역.
decide	전체 흐름을 묶고 최종 action 을 반환한다.

부록 B. 현장 명령어

```
node tools/verify_v4.mjs PikaPlanner_v4.js
node tools/mirror_sanity.mjs PikaPlanner_v4.js
node tools/frozen_round_regression.mjs PikaPlanner_v4.js
node tools/cushion_regression.mjs PikaPlanner_v4.js
node tools/tournament_one.mjs v05 LEFT 1
node tools/tournament_one.mjs v07 RIGHT 1
```

실제 당일 repository 의 테스트 방식이 제공되면 그것을 최우선으로 쓴다. 이 패키지의 simulator 는 후보 비교와 버그 재현용이다.

부록 C. 현재 v4 검증 기록

- JS syntax / 900-call action contract test: 통과.
- 이번 컨테이너 benchmark: 평균 약 3.2ms/call, 최대 약 25~30ms 범위.
- mirror sanity: 1000/1000 safe paired snapshots 에서 action mirror 일치.
- cushion regression: 벽/네트 위/네트 옆 테스트 통과.
- frozen round regression: 52/52 reset snapshots 에서 y=0, hit=0 유지.
- 수정된 tournament harness seed 1 기준 tuned v1, v05, v07 은 양쪽에서 승리하는 회귀 결과를 확인했다.
- v2/v2.1 은 장기 랠리 stress 상대다. 특정 v2 RIGHT stress orientation 은 4 분 기준 0-2 로 패한 사례가 남아 있다. 이는 v4 가 모든 고도화 상대를 압도한다는 뜻이 아니며, 실제 참가자 수준을 예측하는 수치로 쓰지 않는다.

마지막으로 기억할 것

좋은 현장 패치는 “코드를 많이 추가한 패치”가 아니라 “새 스킬 때문에 달라진 물리와 접촉 시간만 정확히 추가하고, 기존 v4 의 안전성을 그대로 보존한 패치”다.